

Assignment: Full-Stack Web Application Development

Task

You are required to design, develop, and deploy a full-stack web application of your choice. The system must include a back-end API built with ASP.NET Core, a front-end built with React, a relational database integrated with EF Core, and unit testing using xUnit.

Your application should model a realistic use case (e.g., library management, student portal, inventory tracker, event booking system, task manager, etc.). You have the freedom to decide on the system, but the following technical requirements are compulsory.

Requirements

1. Back-End with ASP.NET Core

- Create at least **two controllers** (e.g., UserController, ItemController) with CRUD operations.
- Implement **actions with validation** using Data Annotations ([Required], [MaxLength], [EmailAddress] if applicable).
- Ensure at least one action includes **duplicate-check logic** (e.g., no duplicate emails, no duplicate item codes).
- Apply **multiple routing options** (e.g., /api/users/register and /api/register).
- All actions must be **asynchronous** using async/await.

2. Database Integration with EF Core

- Create a DbContext class with at least **two DbSet entities**.
- Apply **code-first migrations** to generate the schema.
- Define at least one **relationship** (e.g., One-to-Many, such as User → Orders, Lecturer → Courses).
- Implement and demonstrate at least **one filtered query** (e.g., return all users with Role = "Student", return all items in stock).

3. Front-End with React

- Create a **form-based component** (e.g., registration form, add item form) with validation.
- Submit form data to the back-end API using **fetch or Axios**.

- Create at least **four pages** (e.g., HomePage, ProfilePage, Dashboard and DetailsPage).
- Implement **navigation between pages** using React Router.
- Apply basic styling (CSS, Bootstrap, Tailwind, or Material UI).

4. Testing with xUnit

- Write at least **two unit tests** for one controller action:
 - A success case (valid data is processed correctly).
 - A failure case (e.g., duplicate entry, missing required field).
- Write at least **one integration test** using an InMemory database to test controller + DbContext together.

5. Deployment & Documentation (10%)

- Prepare the system for **deployment** (IIS, Azure, or local hosting).
- Enable **Swagger/OpenAPI** for API documentation.
- Provide a **README file** that includes:
 - Project setup instructions
 - Database migration steps
 - API endpoints description
- Present a short **demo** of your system.

Marking Rubric

Criterion	Excellent (80–100%)	Good (60–79%)	Satisfactory (50–59%)	Poor (0–49%)
Back-End (35%)	Multiple controllers, full CRUD, validation, routing, async methods	CRUD mostly functional, some validation missing	Minimal CRUD, weak validation	Incomplete controllers
Database (15%)	Proper DbContext, migrations, CRUD, relationships implemented	CRUD working, limited relationships	Basic DbContext, minimal queries	No working DB integration
Front-End (25%)	React app with form, routing, validation, styled UI	React app with form & navigation but weak validation	Minimal React form & routing	No functional React front-end
Testing (15%)	Multiple unit & integration tests, valid + invalid cases	Unit tests only, partial coverage	One simple test	No testing
Deployment & Docs (10%)	Deployable, Swagger enabled, clear README & demo	Deployable locally, README provided	Minimal docs, unclear deployment	No docs or deployment proof